

操作系统

第二章 进程管理

第二部分 (5) 并发与调度 (下)

时钟中断、应用程序时间片轮转和定时器服务

同济大学计算机系

序、Unix V6++进程的优先数

- 现运行进程 SRUN
 - 执行应用程序, ≥ 100
 - 中断处理程序, ≥ 100
 - 系统调用不睡或上半段, ≥ 100
 - SSLEEP系统调用下半段 $[-100, 0]$
 - SWAIT系统调用下半段 $(0, 100)$
- 阻塞进程
 - SSLEEP, < 0
 - SWAIT, > 0
- 就绪进程 SRUN
 - SSLEEP唤醒, $[-100, 0]$
 - SWAIT唤醒, $(0, 100)$
 - 被剥夺的应用程序, ≥ 100

核心态不调度 ∴ 中断处理程序全部执行完毕 & 系统调用全部执行完毕 (返回用户态, 入睡 或 终止) 才会放弃CPU

Unix V6 尽力提升信号接收进程优先权
(p_pri调整至100, 为信号处理子系统提速)



一、Unix V6++ 应用程序轮流使用CPU

使用时间片轮转调度的思路：

- 现运行进程连续使用CPU很久，优先级下降，让出CPU；
- 其余进程，很久未使用CPU优先级上升；
- 让出的CPU给等待最久的进程。

用户态进程，优先数的计算公式

$p_pri = \min(127, \text{静态优先数} + p_cpu/16)$

- 静态优先数 = $PUSER(100) + p_nice$;
 - 前台进程 p_nice 是 0
 - 后台进程 p_nice 是 20
- p_cpu 是进程 CPU 使用时长，用来调节优先级；
 - CPU 使用很久的、 p_cpu 上升、优先权下降；
 - 排队很久的、 p_cpu 下降、优先级上升；
 - p_nice 相同的进程，轮流使用 CPU

后台进程的静态优先数远远高于前台进程 ∴ 有前台进程需要运行，后台进程没有运行机会
算力留给前台进程，系统的响应速度快。

```
class Process
{
    .....
    int p_cpu;        // 衡量进程最近使用CPU的强度
    int p_nice;       // 静态优先数
    int p_pri;        // 进程优先数
    .....
}
```

时钟中断处理程序，用变化的 p_cpu ，驱动应用程序轮流使用 CPU 资源

实现



- 时钟中断，每 $1/\text{HZ}$ 秒一次 $\therefore$ 时钟中断处理程序每 $1/\text{HZ}$ 秒执行一次
- 每次执行，对现运行进程： p_cpu++
- 每个整数秒，所有进程： $p_cpu - \text{SCHMAG}(20)$ ，确保 ≥ 0
- $\text{SetPri}()$ 重算所有进程的 p_pri ，正在执行系统调用的进程 p_pri 不动，信号接收进程 p_pri 尽量不碰
- 若现运行进程优先级下降，执行 $\text{Swch}()$ 让出CPU



情景分析:

假设系统中存在4个CPU bound进程 PA、PB、PC、PD，这些进程静态优先数相等：100，p_cpu初值0。Process[5]、[7]、[8]、[9]分别是PA、PB、PC和PD进程的PCB。T时刻，PA先运行。观察这些进程如何轮流使用CPU。

p_cpu		T	T+1	T+2	T+3	T+4	T+5			
	PA	0	40	20	0	0	40			
	PB	0	0	40	20	0	0			
	PC	0	0	0	40	20	0			
	PD	0	0	0	0	40	20			
p_pri		T	T+1	T+2	T+3	T+4	T+5			
	PA	100	102	101	100	100	102			
	PB	100	100	102	101	100	100			
	PC	100	100	100	102	101	100			
	PD	100	100	100	100	102	101			

SCHMAG = 20
HZ = 60

时钟中断处理程序，代码实现

每次时钟中断 p_cpu++

```
User& u = Kernel::Instance().GetUser();  
Process* current = u.u_procp;  
current->p_cpu = Utility::Min(++current->p_cpu, 127);
```

每秒，调整所有应用程序的优先数，判断现运行进程有否用完时间片

```
for( int i = 0; i < ProcessManager::NPROC; i++ )  
{  
    Process* pProcess = &procMgr.process[i];  
    if ( pProcess->p_stat != Process::SNULL )  
    {  
        if ( pProcess->p_cpu > SCHMAG )  
            pProcess->p_cpu -= SCHMAG;  
        else  
            pProcess->p_cpu = 0;  
        if ( pProcess->p_pri > ProcessManager::PUSER )  
            pProcess->SetPri();  
    }  
}
```

```
if ( (context->xcs & USER_MODE) == USER_MODE )  
    current->SetPri();
```



```
void Process::SetPri()
{
    int priority;
    ProcessManager& procMgr = Kernel::Instance().GetProcessManager();

    priority = this->p_cpu / 16;
    priority += ProcessManager::PUSER + this->p_nice;

    if ( priority > 255 )
    {
        priority = 255;
    }
    if ( priority > procMgr.CurPri )
    {
        procMgr.RunRun++;
    }
    this->p_pri = priority;
}
```

CPU密集型的现运行进程，时钟中断入口
程序判断RunRun标识，大于0，放弃CPU。



二、Unix V6++系统的时间

1、时钟（硬件）

- **RTC（Real Time Clock）实时钟，在主板上，电池驱动。关机时，维护系统时间。**
- **PIT（Programmable Interval Timer）周期性中断时钟。是连在中断控制器上的外设芯片。系统工作时的主时钟。 8254时钟芯片。**
- **TSC（Time Stamp Counter）高精度时钟。是CPU内部的64位寄存器。每个CPU时钟周期（CPU cycle），值加1。**



时钟的用途

- 内核初始化时，读RTC的值，写全局变量 time。time是系统时钟，叫做墙上时间 wall clock time。
- 系统正常工作时，RTC不工作。 PIT为系统提供均匀的脉冲信号。这就是**时钟中断**。
系统对时钟中断计数，调整time变量的值，实施与时间有关的任务。
- TSC，用来实现高精度定时器 + 修正time误差。



2、UNIX V6++ 管理时间的Time对象

```
class Time                                     // Unix系统时钟管理器。整个系统只有一个Time对象
{
    public:
        static const int SCHMAG = 10;          /* p_cpu 的 magic number, 让进程的优先级随排队时间上升 */
        static const int HZ = 60;              /* 赫兹, 每秒时钟中断次数 */

        static int lbolt;                      /* 时钟中断的计数器 */
        static unsigned int time;              /* 墙上时间, 自1970年1月1日至今的秒数 */
        static unsigned int tout;              /* 为应用程序提供计时器服务 */

        static void TimeInterruptEntrance();    /* 时钟中断入口函数, 入口地址登记在 IDT[0x20] */
        static void Clock(struct pt_regs* regs, struct pt_context* context); /* 时钟中断处理函数 */
};
```

1/HZ, 时钟脉冲的间隔。叫做tick (时钟滴答), 是计算机系统的计时单位

3、Clock()要做的工作

- 累加内核计数器（与时间有关的计数器，有好多）

每次时钟中断都要做

- 调整时钟
- 时间片轮转调度，让应用程序轮流使用CPU
- 唤醒延迟睡眠的进程
- 定期执行内核任务

这些耗时的操作，每秒做一次
(若整数秒正在执行内核任务，暂缓至下个时钟滴答)

4、Clock() 调整系统时钟

```
Clock()  
{  
    lbolt++; ...累加其它内核计时器...  
    if(lbolt < HZ) // 60  
        EOI, return;  
    lbolt -= HZ; // 整数秒  
    time++;  
    ...整数秒要执行的其它任务...  
    return  
}
```

```
累加其它内核计时器  
if ( ++Time::lbolt < HZ )      //非整数秒, 返回  
    return;  
/* ----- 整数秒 ----- */  
{  
    Time::lbolt -= HZ;  
    Time::time++; // 数秒  
    整数秒要执行的其它任务  
}
```

5、Clock() 耗时操作，执行时响应所有外设中断

```
Clock()  
{  
    lbolt++; ...累加其它内核计时器...  
    if(lbolt < HZ) // 60  
        EOI, return;  
    lbolt -= HZ; // 整数秒  
    time++;  
    STI, EOI  
    ...整数秒要执行的其它任务...  
    return  
}
```

```
X86Assembly::STI();  
IOPort::OutByte(Chip8259A::MASTER_IO_PORT_1, Chip8259A::EOI);
```



6、如果整数秒时刻正在执行内核任务，耗时任务下个时钟滴答再做无妨

Clock()

```
{  
    lbolt++; ...累加其它内核计时器...  
    if( lbolt < HZ ) // 60  
        EOI, return;  
    if( 时钟中断先前核心态 & 系统不是idle状态 )  
        EOI, return;  
    lbolt -= HZ; // 整数秒  
    time++;  
    STI, EOI  
    ...整数秒要执行的其它任务...  
    return  
}
```

```
if( current->p_stat == Process::SRUN &&  
    (context->xcs & USER_MODE) == KERNEL_MODE )  
{  
    IOPort::OutByte(Chip8259A::MASTER_IO_PORT_1, Chip8259A::EOI);  
    return;  
}
```


7、累加内核计时器

Clock()

```
{  
    lbolt++; ...累加其它内核计时器...  
    if( lbolt < HZ ) // 60  
        EOI, return;  
    if( 时钟中断先前核心态 & 系统不是idle状态 )  
        EOI, return;  
    lbolt -= HZ; // 整数秒  
    time++;  
    STI, EOI  
    ...整数秒要执行的其它任务...  
    return  
}
```

```
if ( (context->xcs & USER_MODE) == USER_MODE )  
    u.u_ftime++; // 当前进程用户态下执行的时间 ++  
else  
    u.u_stime++; // 当前进程核心态下执行的时间 ++  
current->p_cpu = Utility::Min(++current->p_cpu, 1024); // 进程使用CPU时长
```

8、时钟中断的运行情况



每个时钟滴答执行一次。

- 非整数秒，累加计数器返回，没有调度。如遇关中断，开中断后执行时钟中断处理程序。
- 整数秒，正在执行系统调用 或 其它中断处理程序，累加计数器返回，没有调度。
 - 否则，开中断执行耗时操作：维护系统时钟，调度，尝试换入盘交换区上的就绪进程。
 - 系统idle的时候，0#进程执行时钟中断处理程序，维护系统时钟。



9、完整的代码

```
void Time::Clock( struct pt_regs* regs, struct pt_context* context )
{
    User& u = Kernel::Instance().GetUser();
    ProcessManager& procMgr = Kernel::Instance().GetProcessManager();

    if ( (context->xcs & USER_MODE) == USER_MODE )
    {
        u.u_utime++;    // 进程用户态执行时长
    }
    else
    {
        u.u_stime++;    // 进程核心态执行时长
    }

    Process* current = u.u_procp;
    current->p_cpu = Utility::Min(++current->p_cpu, 1024);    // 现运行进程 p_cpu++
}
```



```
if ( ++Time::lbolt < HZ )
{
    /* 对主8259A中断控制芯片发送EOI命令。 */
    IOPort::OutByte(Chip8259A::MASTER_IO_PORT_1, Chip8259A::EOI);
    return;
}
else
{
    /* 到了整数秒末尾，根据先前态决定是否继续执行以下操作*/

    /* 时钟中断打断了内核任务，立即返回。耗时的计算下一次时钟中断再做 */
    /*if( (context->xcs & USER_MODE) == KERNEL_MODE && current->p_pid != 0)*/
    if( current->p_stat == Process::SRUN && (context->xcs & USER_MODE) == KERNEL_MODE )
    {
        IOPort::OutByte(Chip8259A::MASTER_IO_PORT_1, Chip8259A::EOI);
        return;
    }

    Time::lbolt -= HZ;
    Time::time++;
    X86Assembly::STI();
    IOPort::OutByte(Chip8259A::MASTER_IO_PORT_1, Chip8259A::EOI);
}
```



```
if ( Time::time == Time::tout )  
{  
    procMgr.WakeUpAll((unsigned long)&Time::tout); // 唤醒设置定时器入睡的进程  
}
```



```
/* 重算所有进程的p_time, p_cpu,以及优先数p_pri */
for( int i = 0; i < ProcessManager::NPROC; i++ )
{
    Process* pProcess = &procMgr.process[i];
    if ( pProcess->p_stat != Process::SNULL )
    {
        pProcess->p_time = Utility::Min(++pProcess->p_time, 127);

        if ( pProcess->p_cpu > SCHMAG )
        {
            pProcess->p_cpu -= SCHMAG;
        }
        else
        {
            pProcess->p_cpu = 0;
        }

        if ( pProcess->p_pri > ProcessManager::PUSER )
        {
            pProcess->SetPri(); // 重算优先数
        }
    }
}
```

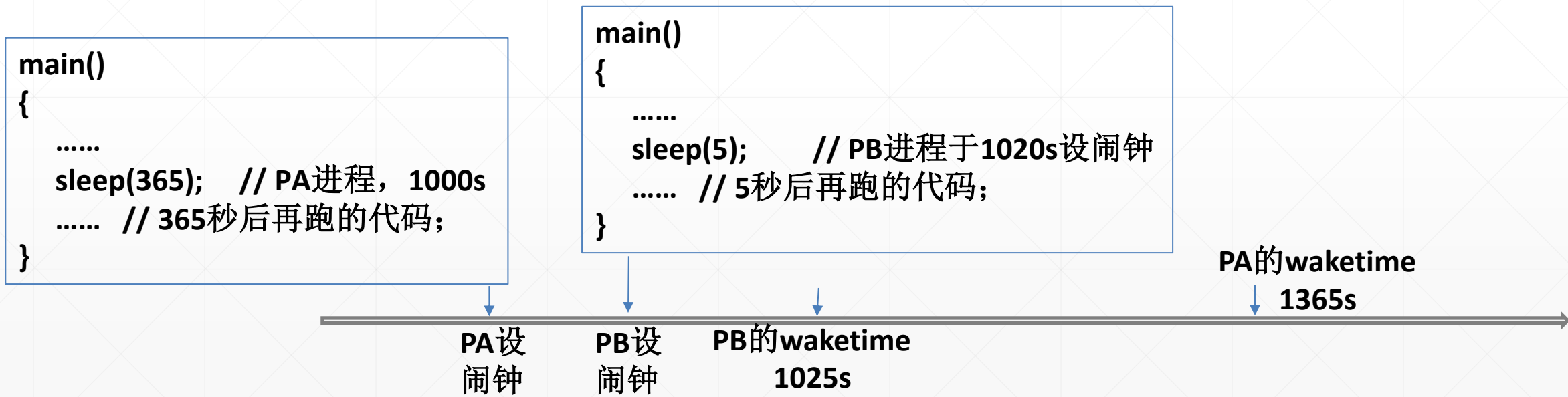


```
if ( procMgr.RunIn != 0 )
{
    procMgr.RunIn = 0;
    procMgr.WakeUpAll((unsigned long)&procMgr.RunIn);
} // 如果盘交换区有进程，每秒1次，尝试将其搬回内存

if ( (context->xcs & USER_MODE) == USER_MODE )
{
    if ( current->IsSig() )
    {
        current->PSig(context);
    }
    current->SetPri();
} // 如果中断前为用户态，处理信号、重算优先数
}
```

10、为应用程序提供定时器服务

- 系统调用 `sleep(seconds)`: 应用程序执行`sleep`系统调用设置定时器。执行它的进程会入睡, 直至 `time + seconds`。之后进程恢复SRUN, 接受CPU调度。这是定时睡眠进程。
- `time + seconds`为进程的waketime。在所有waketime时刻, 系统应该准确唤醒定时器到期的所有进程。



实现

- 1、内核全局变量 **tout**，记录所有进程 **waketime** 的最小值
- 2、每个进程记录自己的 **waketime**（自己的核心栈）
- 3、**tout** 到期的时候，每个进程查看自己的 **waketime**，到期苏醒；不到期，继续睡，重新设置 **tout** 的值

```
main()
{
    .....
    sleep(365); // PA进程，1000s
    ..... // 365秒后再跑的代码;
}
```



sleep(seconds)系统调用: Sys_Sslep() 函数

- 1、计算 **waketime = time + seconds**
- 2、设置 **tout**
 - 没有未到期定时器: **tout = wakeTime**
 - 有 ~: **tout = min(waketime, tout)**
- 3、**sleep(&Time::tout, 90)** 入睡:


```
p_stat = SWAIT
p_pri = 90
p_wchan = &tout
swtch( )
```
- 4、**waketime == tout ?**
 - y: **sleep**系统调用返回，进程回用户态执行后续代码
 - n: **goto 2**

时钟中断处理程序 **clock()**，每个整数秒

```
.....
if ( Time::time == Time::tout )
    procMgr.WakeUpAll(&Time::tout);
.....
```

代码



```
int SystemCall::Sys_Ssleep()    // 系统调用sleep
{
    User& u = Kernel::Instance().GetUser();

    X86Assembly::CLI();
    unsigned int wakeTime = Time::time + u.u_arg[0];           // sleep(seconds)
    while( wakeTime > Time::time )                             // waketime没到, 会再次入睡
    {
        if ( Time::tout <= Time::time || Time::tout > wakeTime )
        {
            Time::tout = wakeTime;
        }
        u.u_procp->Sleep((unsigned long)&Time::tout, ProcessManager::PSLEP);
    }
    X86Assembly::STI();

    return 0;          /* GCC likes it ! */
}
```